

A Reproducible Software Workflow for Compact Pauli Analysis of Molecular Vibrational Hamiltonians

Saravana Prakash thirumuruganandham ^{1, a)} Eduardo Patricio Estevez Ruiz ^{2, b)} and Danny Raúl Argüero Zapata¹

¹⁾ *Research and Development, SIT HEALTH SAS, Catalina Aldaz, Quito, Ecuador*

²⁾ *Escuela de Ciencias Físicas y Nanotecnología, Universidad de Investigación de Tecnología Experimental Yachay, Hacienda San José S/N, 100119 Urcuquí, Ecuador*

(Dated: 30 September 2026)

Molecular vibrational calculations become increasingly expensive as molecular size increases, while quantum computing provides compact representations of these problems using qubits. In this work, we present a reproducible software workflow that connects classical normal-mode analysis with compact Pauli representations of molecular vibrational Hamiltonians. The workflow integrates normal-mode calculations, power-of-two encoding, Pauli decomposition, Hamiltonian compression, Suzuki–Trotter propagation, and computational resource monitoring. For ten molecular benchmark systems spanning 51–201 Cartesian degrees of freedom, the Hamiltonians are represented using only 6–8 qubits, while full Pauli reconstruction reproduces the original operators with relative errors of approximately 10^{-16} . We find that retaining most of the Pauli-coefficient weight does not necessarily preserve the vibrational spectrum, demonstrating that compression must be evaluated using physical observables rather than coefficient retention alone. The workflow is further applied to proteins spanning 912–5568 Cartesian degrees of freedom. Projection onto a 64-dimensional modal subspace produces six-qubit reduced Hamiltonians, while the classical computational cost continues to increase with protein size. These results demonstrate that molecular-system size, compact quantum representation, and classical computational cost are distinct quantities that should be evaluated separately.

Keywords: Normal-mode analysis, Pauli decomposition, Molecular vibrational Hamiltonians, Suzuki–Trotter propagation, Quantum chemistry algorithms, Compact binary encoding

I. INTRODUCTION

Electronic-structure methods become broadly useful only when their underlying theory is translated into software that is numerically reliable, reproducible, and practical to use. Scientific software must therefore be assessed not only by the physical models it implements, but also by its numerical accuracy, computational scaling, resource requirements, and reproducibility¹. These considerations become especially important when results obtained from conventional molecular calculations are transformed into representations intended for quantum algorithms^{2, 3, 4}.

Molecular vibrations provide a natural setting in which to examine this transition. Within the harmonic approximation, normal-mode analysis (NMA) describes vibrational motion through the eigensolution of a mass-weighted Hessian⁵. The method is well established, but constructing and diagonalizing the full Hessian becomes increasingly demanding as the molecular system grows^{6, 7}. This limitation is particularly relevant for proteins, where low-frequency collective motions are often of greater interest than the complete vibrational spectrum^{8, 9, 10}. Partial-Hessian, reduced-space, coarse-grained, and iterative approaches have therefore been developed to focus computational effort on the physically relevant portion of the spectrum^{11, 12}. More recent matrix-free strategies further

show that selected low-frequency protein modes can be obtained without explicitly constructing the complete Hessian¹³.

A related question arises when a vibrational Hamiltonian is prepared for quantum computation: how compactly can the molecular problem be represented without obscuring the computational work required to obtain that representation? A finite-dimensional real symmetric operator can be embedded in a power-of-two space and expanded in tensor products of Pauli operators. For the compact binary encoding considered here, a problem of dimension d requires only $n_q = \lceil \log_2 d \rceil$ index qubits. This logarithmic growth makes compact encoding attractive for connecting conventional vibrational calculations with quantum-compatible Hamiltonians. Pauli representations and product-formula methods such as Suzuki–Trotter decompositions provide established tools for this purpose^{14, 15, 16}, and related approaches are increasingly being explored for molecular vibrational problems^{17, 18, 19}. However, a small qubit register describes the size of the encoded space; it does not by itself imply a proportionally smaller Hamiltonian, lower classical preprocessing cost, or an end-to-end quantum speedup.

This distinction becomes equally important once the Pauli representation is compressed. Removing terms according to coefficient magnitude can reduce the operator representation, but it can also modify eigenvalues, rotate eigenvectors, and alter the observables derived from them. A spectral difference observed after time propagation therefore cannot automatically be identified as Suzuki–Trotter error: part, or even most, of that differ-

^{a)} Electronic mail: saravana@sit.health

^{b)} Electronic mail: eduardoepr45@gmail.com

ence may already have entered when the Hamiltonian was sparsified. Product-formula errors are also known to depend on the structure and noncommutativity of the Hamiltonian rather than solely on its nominal size. For this reason, encoding, compression, and propagation need to be tested as separate numerical stages.

The workflow developed here follows that principle. It first establishes a classical NMA reference and verifies the complete Pauli reconstruction before any terms are removed. Controlled sparsification is then applied, after which Suzuki–Trotter propagation is compared with the exact evolution of the *same retained Hamiltonian*. This ordering makes it possible to identify whether an observed discrepancy originates in the eigensolution, compact mapping, Hamiltonian compression, or product-formula approximation. Runtime and resident-set-size (RSS) measurements are recorded separately for the main computational stages so that representational compactness is not inadvertently reported as a reduction in computational cost.

We first apply this framework to ten molecular benchmarks spanning 51–201 Cartesian degrees of freedom, corresponding to compact registers of 6–8 qubits. The study is then extended to proteins in order to examine what happens when the physical molecular dimension becomes substantially larger. The protein set spans 912–5568 Cartesian degrees of freedom, for which direct compact indexing of the complete Cartesian spaces would require 10–13 qubits. In the protein workflow, however, the Hamiltonian is projected onto a fixed modal subspace with $D_{\text{red}} = 64$, giving a six-qubit reduced representation for every system considered. These six qubits encode the selected 64-dimensional modal subspace, not the complete protein Hessian; whether that subspace is sufficient for a particular physical observable must be established independently through rank-convergence analysis.

The protein extension therefore provides a useful test of a distinction that is less visible in the smaller molecular systems. Physical system size, encoded Hilbert-space dimension, and classical computational cost are related, but they are not interchangeable measures of complexity. A reduced modal Hamiltonian can remain compact while the work required to construct and validate it continues to increase with the size of the underlying protein. This separation is consistent with the broader motivation for reduced and iterative treatments of macromolecular normal modes^{???} and provides the basis for the protein-scale analysis presented below.

The resulting software is intended as a transparent validation and representation framework rather than as a general electronic-structure engine. Its main contributions are a modular separation of eigensolver, encoding, sparsification, and product-formula errors; exact validation of the Pauli mapping before compression; controlled sparsification and same-Hamiltonian Suzuki–Trotter tests; stage-resolved runtime and memory telemetry; and a fixed-rank protein-scale study that separates physical molecular size from encoded dimension and classical computational

cost. The current MMFF94s backend is used to test the numerical workflow rather than to provide final *ab initio* spectroscopy. Because the numerical core operates on a real symmetric vibrational Hamiltonian, the same validation strategy can subsequently be connected to electronic-structure Hessians and physical dipole derivatives.

II. SOFTWARE SCOPE AND ARCHITECTURE

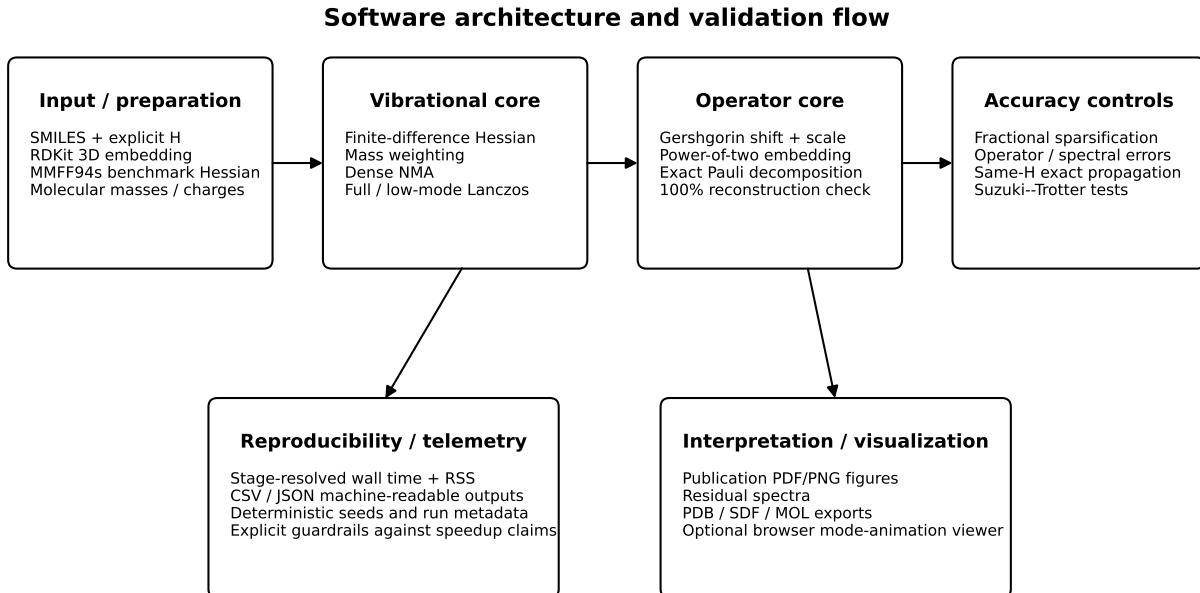
A. Implemented workflow

The reference implementation is organized as a sequence of independently testable stages rather than a single monolithic calculation (Fig. 1). The input/preparation layer creates a hydrogen-complete molecular structure, performs a deterministic 3D embedding, optimizes the geometry, and evaluates gradients needed for the finite-difference Hessian. The vibrational layer constructs the mass-weighted Hessian and computes Dense, full-Lanczos, and low-mode iterative references. The operator layer normalizes and pads the physical matrix, performs the exact Pauli decomposition, and verifies that the 100% reconstruction returns the input operator. The accuracy-control layer carries out coefficient-ranked sparsification and same-Hamiltonian exact/Trotter propagation. Finally, the reporting layer writes all numerical metrics, plots, structures, and viewer files.

This staging is deliberate. A failed result can be localized to a specific boundary: the classical eigensolution, the compact mapping, the retained Pauli operator, or the product formula. In practice, that separation was more useful during development than a single aggregate accuracy score.

B. Command-line interface and machine-readable outputs

The code is operated from a Python command line. User-selectable settings include the molecular benchmark subset, finite-difference displacement, Pauli-retention fractions, Trotter-retention fractions, product-formula step counts, random seed, and viewer options. Each molecule receives its own output directory containing the optimized structure, Cartesian and mass-weighted operators, mode tables, compression metrics, Trotter metrics, stage-resolved runtime/RSS telemetry, summary metadata, and publication figures. The principal files are `summary.json`, `full.pauli.validation.json`, `full.modes.dense.lanczos.csv`, `pauli.compression.csv`, `trotter.convergence.csv`, `spectrum.curves.csv`, and `runtime.memory.stages.csv`. Keeping these intermediate products visible makes it possible to audit a figure against the underlying numerical table rather than relying on a single rendered plot.



The current reference backend is MMFF94s; a production ESS2.0 submission should add at least one tested electronic-structure Hessian/dipole interface.

FIG. 1. Software architecture and validation flow. The current reference backend performs molecular preparation and MMFF94s Hessian construction, after which the numerical core validates Dense/Lanczos eigensolutions, constructs the compact Pauli representation, performs controlled sparsification, and evaluates exact and Suzuki–Trotter propagation of the same retained operator. Telemetry and visualization are kept as separate output layers. A production Electronic Structure Software 2.0 version should connect the same numerical core to one or more tested electronic-structure Hessian and dipole-derivative interfaces.

C. Design decisions motivated by failure modes

Three implementation decisions are central to the current version. First, Pauli compression is specified by a retained fraction rather than a fixed number of terms. This makes the requested approximation comparable when the padded operator dimension changes. Second, the exact 100% Pauli reconstruction is mandatory before a compression sweep. A compressed-spectrum error is therefore not allowed to hide a mapping error. Third, Suzuki–Trotter error is evaluated only against the exact exponential of the same compressed Hamiltonian. The program separately reports error relative to Dense NMA so that compression and propagation errors remain distinguishable.

The code also avoids using a dense eigenspectrum to normalize the operator before the compact transformation. Instead, a Gershgorin bound and infinity norm provide a deterministic shift-and-scale procedure. This choice prevents the supposedly compact stage from depending on information available only after a complete dense diagonalization.

D. Reproducibility and software-engineering scope

The present prototype emphasizes inspectability over aggressive optimization. Randomized molecular embedding uses an explicit seed; each stage is timed with

`time.perf_counter`; RSS is sampled with `psutil`; the backend and dimensions are recorded in summary files; and numerical guardrails are printed directly in the console output. The implementation reuses established community libraries (NumPy, SciPy, pandas, RDKit, psutil, Matplotlib, and optional browser libraries for visualization) rather than reimplementing basic linear algebra or molecular file handling. This modular approach is consistent with the software-ecosystem model emphasized in previous JCP software collections.^{1,2}

The current code is a research prototype rather than a mature general-purpose package. Before journal submission, the public repository should contain an installable environment specification, automated smoke tests, at least one reproducible example input/output pair, a versioned release, and a tested electronic-structure import path. These are not cosmetic additions: JCP software referees may be asked to install and assess the program itself.²

III. THEORY AND ALGORITHMS

A. Mass-weighted harmonic vibrational problem

[?] For Cartesian coordinates $\mathbf{x} \in \mathbb{R}^{3N}$, the Cartesian Hessian is

$$(\mathbf{H}_{\text{cart}})_{ij} = \frac{\partial^2 V}{\partial x_i \partial x_j}. \quad (1)$$

In the implementation used here, Hessian columns are obtained by a central finite difference of force-field gradients,

$$(\mathbf{H}_{\text{cart}})_{:,j} \approx \frac{\mathbf{g}(\mathbf{x} + \epsilon \mathbf{e}_j) - \mathbf{g}(\mathbf{x} - \epsilon \mathbf{e}_j)}{2\epsilon}, \quad (2)$$

with $\epsilon = 10^{-3}$ Å. The mass-weighted Hessian is

$$\mathbf{H}_{\text{mw}} = \mathbf{M}^{-1/2} \mathbf{H}_{\text{cart}} \mathbf{M}^{-1/2}, \quad (3)$$

where each atomic mass is repeated for its three Cartesian components. Dense NMA solves

$$\mathbf{H}_{\text{mw}} \mathbf{v}_k = \lambda_k \mathbf{v}_k. \quad (4)$$

The code retains the sign of negative eigenvalues to flag imaginary modes and converts eigenvalues to wavenumbers as

$$\tilde{\nu}_k = \text{sgn}(\lambda_k) \frac{1}{2\pi c} \sqrt{|\lambda_k| \frac{4184/N_A}{10^{-20} m_u}}, \quad (5)$$

for Hessian units of $(\text{kcal mol}^{-1})/(\text{\AA}^2 \text{ amu})$. The six modes with the smallest $|\tilde{\nu}|$ are treated as external translation/rotation modes rather than blindly deleting the first six sorted eigenvalues.

For the present proof-of-concept intensity model, normalized Gasteiger charges q_a are used to construct a fixed-charge dipole-displacement proxy,¹⁴

$$I_k \propto \left\| \sum_{a=1}^N q_a \frac{\mathbf{v}_{ak}}{\sqrt{m_a}} \right\|^2. \quad (6)$$

The stick spectrum is broadened with a Gaussian of width $\sigma = 12 \text{ cm}^{-1}$. Equation (6) is an algorithm-development proxy, not a substitute for a physical electronic-structure dipole derivative. This limitation is addressed explicitly in Sec. VID.

B. Dense, Lanczos, and low-mode Krylov references

^{8?} Dense NMA uses a symmetric eigensolver. Independently, a fully reorthogonalized Lanczos procedure constructs

$$\mathcal{K}_m(\mathbf{H}_{\text{mw}}, \mathbf{q}_1) = \text{span}\{\mathbf{q}_1, \mathbf{H}_{\text{mw}} \mathbf{q}_1, \dots, \mathbf{H}_{\text{mw}}^{m-1} \mathbf{q}_1\}, \quad (7)$$

with double reorthogonalization and a tridiagonal Ritz problem. For the full validation, m is allowed to approach d . A sparse symmetric eigensolver is also used for a selected set of low-frequency modes. This three-level classical reference is useful because a perfect Pauli reconstruction is not meaningful if the classical reference itself is not converged.

C. Shift, scale, and compact power-of-two embedding

The Cartesian dimension d is embedded in

$$D = 2^{n_q}, \quad n_q = \lceil \log_2 d \rceil. \quad (8)$$

No dense eigenspectrum is required to normalize the operator. Instead, a Gershgorin lower bound

$$\ell = \min_i \left[H_{ii} - \sum_{j \neq i} |H_{ij}| \right] \quad (9)$$

is used to define a nonnegative shift

$$s = \max(0, -\ell + 10^{-10}), \quad (10)$$

and a scale

$$a = \|\mathbf{H}_{\text{mw}} + s\mathbf{I}\|_{\infty}. \quad (11)$$

The normalized physical block is

$$\mathbf{H}_n = \frac{\mathbf{H}_{\text{mw}} + s\mathbf{I}}{a}. \quad (12)$$

The remaining padded states are assigned an identity block, giving $\tilde{\mathbf{H}} \in \mathbb{R}^{D \times D}$. Physical eigenvalues are recovered from normalized eigenvalues $\tilde{\lambda}$ using

$$\lambda = a\tilde{\lambda} - s. \quad (13)$$

D. Exact Pauli decomposition

? Any $D = 2^{n_q}$ Hermitian matrix admits

$$\tilde{\mathbf{H}} = \sum_{j=1}^P c_j P_j, \quad P_j \in \{I, X, Y, Z\}^{\otimes n_q}, \quad (14)$$

with

$$c_j = \frac{1}{D} \text{Tr}(P_j \tilde{\mathbf{H}}). \quad (15)$$

The implementation uses binary symplectic x and z masks and stores each nonzero term as two 32-bit integer masks and one 64-bit real coefficient. Terms are sorted by decreasing $|c_j|$. For the real symmetric matrices considered here, the observed number of nonzero stored terms follows $P = D(D+1)/2$ for the tested dimensions, yielding 2080, 8256, and 32896 terms for $D = 64, 128$, and 256, respectively.

The full reconstruction is validated before any truncation using

$$\epsilon_H^{(100)} = \frac{\left\| \tilde{\mathbf{H}} - \sum_j c_j P_j \right\|_F}{\left\| \tilde{\mathbf{H}} \right\|_F}. \quad (16)$$

This is the principal encoding sanity check.

E. Pauli sparsification and physical-subspace leakage

For a requested retained term fraction f , the $K = \lceil fP \rceil$ largest coefficients are retained,

$$\tilde{\mathbf{H}}_f = \sum_{j=1}^K c_j P_j. \quad (17)$$

In addition to term fraction, two cumulative coefficient weights are reported,

$$W_1(K) = \frac{\sum_{j \leq K} |c_j|}{\sum_j |c_j|}, \quad (18)$$

$$W_2(K) = \frac{\sum_{j \leq K} |c_j|^2}{\sum_j |c_j|^2}. \quad (19)$$

Operator error is

$$\epsilon_H(f) = \frac{\|\tilde{\mathbf{H}} - \tilde{\mathbf{H}}_f\|_F}{\|\tilde{\mathbf{H}}\|_F}. \quad (20)$$

Because truncation may couple the physical Cartesian block to padded basis states, eigenstates are ranked by physical-subspace weight

$$w_k = \sum_{i=1}^d |V_{ik}|^2, \quad (21)$$

and the d states with largest w_k are projected back into the physical block and renormalized. Mean and minimum physical-state weights, together with cross-block norms, quantify leakage.

F. Exact and Suzuki–Trotter propagation

For a fixed compressed Hamiltonian $\tilde{\mathbf{H}}_f$, the exact classical propagator is

$$U_{\text{ex}}(t) = \exp(-i\tilde{\mathbf{H}}_f t). \quad (22)$$

The second-order symmetric Suzuki formula used in the code is^{15,16}

$$U_{\text{ST2}}(t; r) = \left[\prod_{j=1}^K e^{-ic_j P_j \Delta t/2} \prod_{j=K}^1 e^{-ic_j P_j \Delta t/2} \right]^r, \quad \Delta t = t/r. \quad (23)$$

Since $P_j^2 = I$,

$$e^{-i\theta P_j} = \cos \theta I - i \sin \theta P_j. \quad (24)$$

An adaptive evolution time is chosen so that exact eigenphases remain inside the principal interval and phase wrapping is avoided. Product-formula error is evaluated only against U_{ex} for the *same* $\tilde{\mathbf{H}}_f$.

The unitary metrics are

$$\epsilon_U = \frac{\|U_{\text{ST2}} - U_{\text{ex}}\|_F}{\|U_{\text{ex}}\|_F} \quad (25)$$

and the phase-insensitive trace fidelity

$$F_{\text{tr}} = \frac{1}{D} |\text{Tr}(U_{\text{ex}}^\dagger U_{\text{ST2}})|. \quad (26)$$

G. Spectral accuracy metrics

For M physical vibrational modes, the frequency mean absolute error (MAE) and root mean square error (RMSE) are

$$\text{MAE}_\nu = \frac{1}{M} \sum_{k=1}^M |\nu_k - \nu_k^{\text{ref}}|, \quad (27)$$

$$\text{RMSE}_\nu = \sqrt{\frac{1}{M} \sum_{k=1}^M (\nu_k - \nu_k^{\text{ref}})^2}. \quad (28)$$

For broadened spectra S and S_{ref} , cosine similarity is

$$C_{\text{spec}} = \frac{S \cdot S_{\text{ref}}}{\|S\| \|S_{\text{ref}}\|}, \quad (29)$$

and the integrated absolute spectral error is

$$E_{\text{int}} = \frac{\int |S(\nu) - S_{\text{ref}}(\nu)| d\nu}{\int |S_{\text{ref}}(\nu)| d\nu}. \quad (30)$$

H. Computational cost and memory model

The central-difference Hessian in Eq. (2) requires $2d$ gradient evaluations. Its cost is therefore approximately

$$T_{\text{Hess}} \sim 2d T_{\text{grad}}. \quad (31)$$

Dense diagonalization has asymptotic cost $\mathcal{O}(d^3)$ and Hessian storage

$$M_{\text{dense}} = 8d^2 \text{ bytes} \quad (32)$$

for double precision. A matrix-free m -step Lanczos process would require m Hessian-vector products and $\mathcal{O}(dm)$ basis storage; the current small-molecule validation stores the dense Hessian and therefore should not be presented as a matrix-free memory advantage.

The direct Pauli decomposition implemented here evaluates D^2 candidate strings with $\mathcal{O}(D)$ coefficient work and is therefore an $\mathcal{O}(D^3)$ classical preprocessing step. With 16 bytes per structured Pauli record,

$$M_{\text{Pauli}} \approx 16K \text{ bytes}. \quad (33)$$

A dense classical complex unitary requires

$$M_U = 16D^2 \text{ bytes}. \quad (34)$$

Finally, the classical emulator applies each Pauli exponential to a dense $D \times D$ unitary at $\mathcal{O}(D^2)$ cost. The approximate work of the second-order implementation is consequently

$$W_{\text{ST2}} \propto 2rKD^2. \quad (35)$$

Equation (35) is a *classical emulation cost*, not a quantum gate-count estimate. Quantum hardware cost would additionally depend on Pauli weight, synthesis, connectivity, fault-tolerance assumptions, state preparation, and measurement.^{17,18}

IV. COMPUTATIONAL DETAILS

Ten toxin-related molecules were used as chemically varied algorithmic benchmarks: Anatoxin-a, Aflatoxin B1, Saxitoxin, Tetrodotoxin, Ricinine, Patulin, Ochratoxin A, Zearalenone, T-2 toxin, and Citrinin. The selection is used only for computational spectroscopy and representation testing. No synthesis, preparation, acquisition, or handling protocol is part of this work.

Hydrogens were added explicitly. Initial 3D coordinates were generated with RDKit ETKDG and optimized with MMFF94s; UFF is available as a fallback in the code, although all ten preflight structures reported MMFF94s parameter coverage.¹¹ The final benchmark code used Python 3.11, NumPy, SciPy, pandas, psutil, and RDKit. Before submission, the manuscript and repository should report the exact package versions from the final reproducible environment rather than a mixture of development runs. Runtime was measured with `time.perf_counter`; resident-set-size (RSS) memory was sampled immediately before and after each stage with psutil. Because RSS deltas do not equal true peak allocation for short-lived temporaries, both analytic representation sizes and measured process RSS are reported.

V. SOFTWARE VALIDATION AND ILLUSTRATIVE APPLICATIONS

The following calculations are intended as software-validation cases. They test whether the implementation preserves the input operator and separates the numerical errors it is designed to diagnose. Because the current benchmark backend is MMFF94s with a fixed-charge intensity proxy, the examples should not be read as final electronic-structure predictions.

A. Full-Hamiltonian validation

The most important numerical control is the 100% Pauli reconstruction. Four independently available validation runs are summarized in Table II. Relative Frobenius errors are in the 10^{-16} range and the broadened spectral cosine

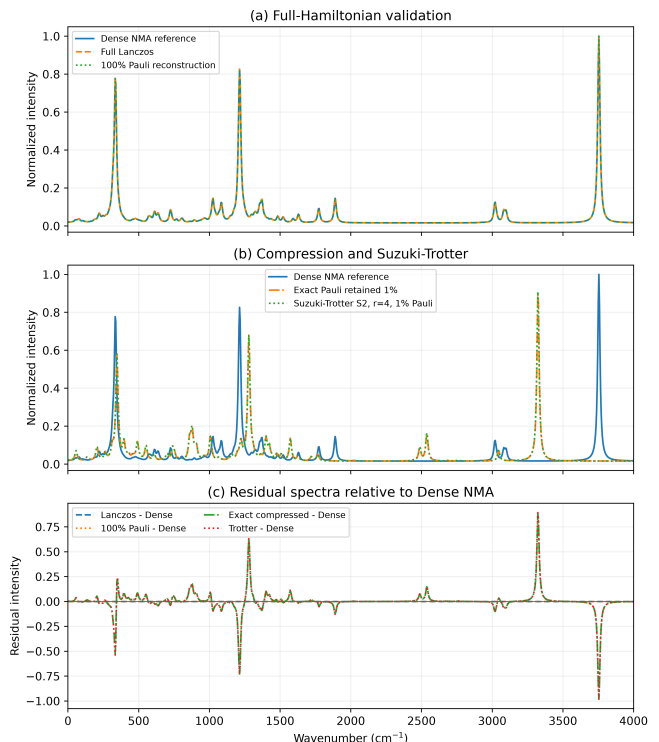


FIG. 2. Zearalenone validation spectrum. (a) Dense NMA, full Lanczos, and 100% Pauli reconstruction overlap across the full spectrum. (b) An extreme 1% Pauli-retention stress test substantially changes the spectrum relative to Dense NMA, while the exact compressed and Suzuki–Trotter spectra remain closely paired. (c) Residuals relative to Dense NMA show negligible differences for Lanczos and the full Pauli reconstruction but large, nearly coincident residuals for the compressed and Trotter results, identifying compression as the dominant source of spectral distortion.

is unity to the reported precision. These results show that the power-of-two embedding and Pauli coefficient construction are numerically lossless before sparsification.

Representative Zearalenone results are shown in Fig. 2. In panel (a), the Dense NMA reference, full Lanczos result, and 100% Pauli reconstruction are visually indistinguishable across the spectrum, confirming that neither the eigensolver cross-check nor the exact Pauli mapping introduces an appreciable spectral distortion. In panel (b), retaining only 1% of the Pauli terms produces pronounced changes in peak positions and intensities relative to Dense NMA. The exact evolution of the compressed Hamiltonian and its second-order Suzuki–Trotter approximation nevertheless remain closely superimposed. The residuals in panel (c) make this separation explicit: the Lanczos–Dense and full-Pauli–Dense residuals remain near zero, whereas the compressed-Hamiltonian residuals are large and are closely reproduced by the Trotter result. Figure 2 therefore localizes the dominant spectral error to aggressive Hamiltonian compression rather than to the exact Pauli mapping or to the subsequent product-formula

TABLE I. Ten-molecule benchmark and compact register size. “Reduction” compares the compact index register $n_q = \lceil \log_2 d \rceil$ with a hypothetical one-qubit-per-Cartesian-DOF representation; it is *not* a classical RAM reduction.

Molecule	Atoms	$d = 3N$	$3N - 6$	n_q	D	Full Pauli terms	Register reduction (%)
Anatoxin-a	27	81	75	7	128	8256	91.36
Aflatoxin B1	35	105	99	7	128	8256	93.33
Saxitoxin	38	114	108	7	128	8256	93.86
Tetrodotoxin	39	117	111	7	128	8256	94.02
Ricinine	20	60	54	6	64	2080	90.00
Patulin	17	51	45	6	64	2080	88.24
Ochratoxin A	46	138	132	8	256	32896	94.20
Zearalenone	45	135	129	8	256	32896	94.07
T-2 toxin	67	201	195	8	256	32896	96.02
Citrinin	32	96	90	7	128	8256	92.71

propagation.

B. Pauli compression defines the accuracy–resource frontier

Patulin provides a complete numerical example of the compression metrics currently available from the final code. Table III shows that the full representation is exact, whereas aggressive sparsification produces large frequency and spectral errors. Particularly noteworthy is the mismatch between coefficient-norm retention and spectroscopic fidelity. At 10% term retention, $W_2 = 0.9951$, yet the operator relative Frobenius error is 0.0698, frequency MAE is 159.2 cm^{-1} , and spectral cosine is only 0.514. Figure 3 extends this comparison over a denser retention sweep. Panel (a) shows that spectral fidelity improves non-linearly with the retained term fraction and approaches the $C_{\text{spec}} = 0.95$ screening level only at comparatively high retention. Panel (b) shows the corresponding systematic decrease in frequency MAE as additional Pauli terms are restored. Panel (c) further shows that a small discarded coefficient weight does not translate directly into a comparably small operator error over the compressed range; only the full representation returns to the machine-precision reconstruction limit. Thus, retaining most of the squared coefficient norm is not sufficient to guarantee preservation of the eigenvalues, eigenvectors, and intensities controlling the broadened spectrum.

These data motivate defining a minimum accepted fraction

$$f_{\min} = \min_f \{f : C_{\text{spec}}(f) \geq C_0, \text{MAE}_\nu(f) \leq \delta_\nu, \epsilon_H(f) \leq \delta_H\} \quad (36)$$

Rather than assuming a universal 1% or 10% representation, Eq. (36) creates a molecule-specific accuracy–resource frontier. A threshold such as $C_0 = 0.95$ can

TABLE II. Independent full-Pauli reconstruction checks from the current validation outputs.

Molecule	d	P	$\epsilon_H^{(100)}$
Aspirin	63	2080	4.36×10^{-16}
Caffeine	72	8256	5.11×10^{-16}
Curcumin	141	32896	8.26×10^{-16}
Patulin	51	2080	3.12×10^{-16}

TABLE III. Patulin Pauli-compression stress test.

Terms (%)	W_2	ϵ_H	$\text{MAE}_\nu \text{ (cm}^{-1}\text{)}$	C_{spec}
100.00	1.0000	3.12×10^{-16}	1.98×10^{-12}	1.000
10.00	0.9951	0.0698	159.18	0.514
1.01	0.9631	0.1920	350.52	0.114

be used as a prespecified screening criterion, with the frequency and operator tolerances chosen according to the intended spectroscopy application.

C. Same-Hamiltonian Suzuki–Trotter validation

The product-formula test is deliberately separated from Eq. (36). For the Patulin 1.01% stress test (21 of 2080 terms), Fig. 4 compares the second-order Suzuki–Trotter propagator directly with the exact exponential of the *same* compressed Hamiltonian, so the plotted quantities contain no compression error. Increasing the number of second-order Trotter steps from $r = 1$ to $r = 2$ reduces the same-Hamiltonian unitary relative Frobenius error from approximately 1.8×10^{-4} to 4.5×10^{-5} , i.e., by about a factor of four. This behavior is consistent with the expected second-order convergence over this two-point test. The trace infidelity, $1 - F_{\text{tr}}$, remains much smaller than the unitary Frobenius error for both step counts. Thus, the Trotter approximation is not exact, but its error is small and decreases rapidly with r ; in the spectral stress tests it remains secondary to the much larger distortion introduced by the 1.01% Hamiltonian compression. These results should not be generalized to denser or more strongly noncommuting Pauli expansions without a broader step-convergence study.

The same error separation is reproduced for Citrinin and T-2 toxin in Fig. 5. In the upper panels, Dense NMA, full Lanczos, and the 100% Pauli reconstruction overlap across the spectrum, again validating the exact mapping. In the middle panels, the 1%–1.01% retained Hamiltonians alter multiple peak positions and intensities relative to Dense NMA, while the exact-compressed and Suzuki–Trotter curves remain nearly superimposed. The lower residual panels show the same structure: Lanczos–Dense and full-Pauli–Dense differences are negligible on

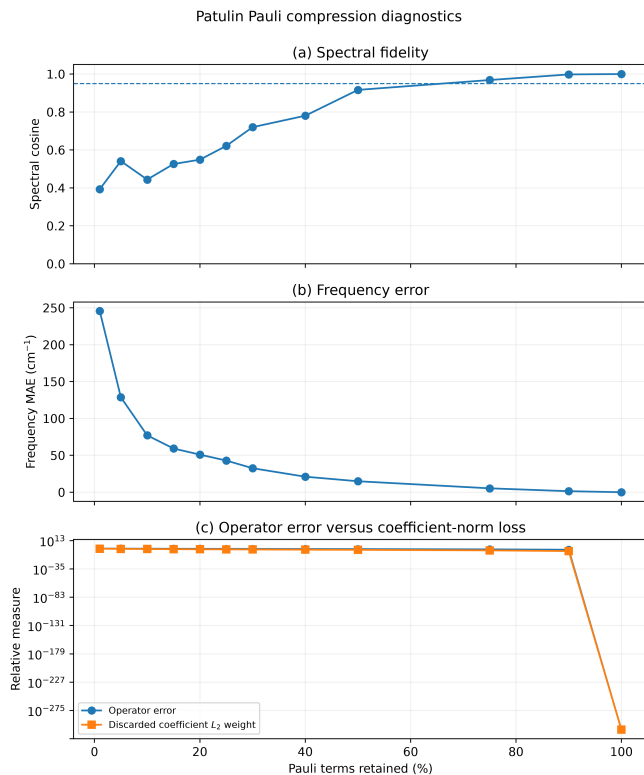


FIG. 3. Patulin compression diagnostics over a dense Pauli-retention sweep. (a) Spectral cosine similarity increases nonlinearly as more Pauli terms are retained; the dashed line marks the $C_{\text{spec}} = 0.95$ screening criterion. (b) Frequency MAE decreases as the retained fraction increases. (c) Operator error is compared with the discarded squared-coefficient weight on a logarithmic scale, showing that coefficient-weight retention alone is not a sufficient proxy for operator or spectroscopic accuracy. The 100% reconstruction returns to the machine-precision limit.

the plotted scale, whereas the compressed and Trotter residuals are large and closely coincident. The fact that this pattern appears in two chemically distinct benchmark molecules supports the interpretation that the dominant error in these extreme stress tests originates from sparsification rather than from the exact Pauli transformation or the subsequent Suzuki–Trotter propagation.

D. Runtime and memory: what is actually reduced?

Figure 6 distinguishes compact register size from classical representation memory. Across the ten-molecule set, d increases from 51 to 201 while n_q remains between 6 and 8. Relative to a one-qubit-per-DOF bookkeeping baseline, the index-register reduction is 88.2–96.0%. This is a genuine representation statement. It does *not*, however, imply lower classical RAM in the present emulator.

For example, the T-2 toxin dense 201×201 double-precision Hessian requires approximately 0.308 MiB,

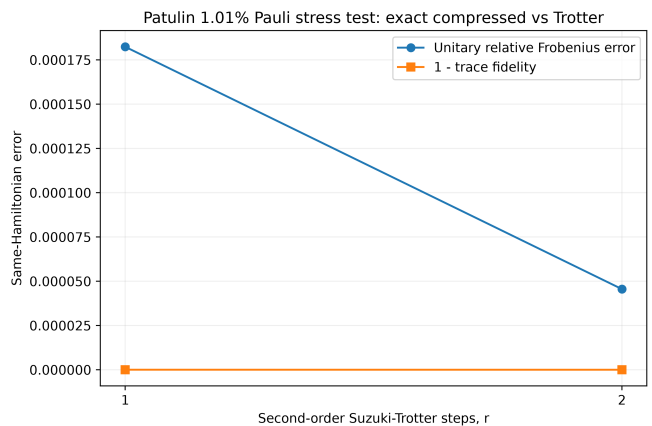


FIG. 4. Same-Hamiltonian Suzuki–Trotter validation for the Patulin 1.01% stress test. Errors are measured against the exact exponential of the same 21-term compressed Pauli Hamiltonian and therefore exclude compression error. Increasing the second-order Suzuki–Trotter step count from $r = 1$ to $r = 2$ reduces the unitary relative Frobenius error by approximately a factor of four, while the trace infidelity remains substantially smaller.

whereas a full 32896-record Pauli list requires approximately 0.502 MiB under the 16-byte record layout, and a dense complex 256×256 unitary requires 1.0 MiB. Thus the full Pauli transformation can increase classical storage at these dimensions. Classical storage becomes competitive only after enough Pauli terms are removed, and that reduction must be constrained by Eq. (36). This result is important because it prevents compact qubit count from being misreported as a classical memory saving.

Stage-resolved timing for the Patulin smoke test is shown in Fig. 7. On the test environment, 3D preparation required 0.0208 s, the 51-column finite-difference Hessian 0.0186 s, Dense NMA 0.00058 s, full Lanczos 0.00119 s, full Pauli decomposition 0.0399 s, and full Pauli reconstruction 0.0334 s. The 10% and 1% reconstructions required 0.00348 and 0.00050 s, respectively. For the 21-term 1% Hamiltonian, second-order Trotter emulation required 0.00135 s at $r = 1$ and 0.00255 s at $r = 2$. These timings demonstrate two points. First, for small molecules, classical Dense NMA is already extremely fast, so speedup claims are neither required nor supported. Second, the current direct Pauli decomposition is itself a nontrivial classical preprocessing cost. The scientifically useful quantity is therefore the accuracy–resource tradeoff, not an asserted wall-time advantage.

Measured process RSS for the same Patulin run increased from approximately 200 MiB before molecule preparation to approximately 238 MiB by the end of the Trotter tests. This process-level memory is dominated by the Python/RDKit/SciPy runtime and transient arrays and therefore should not be confused with the analytic storage requirements in Eqs. (32)–(34). For reproducibility, the software records stage-wise time, RSS before and after the operation, and RSS delta in a machine-readable

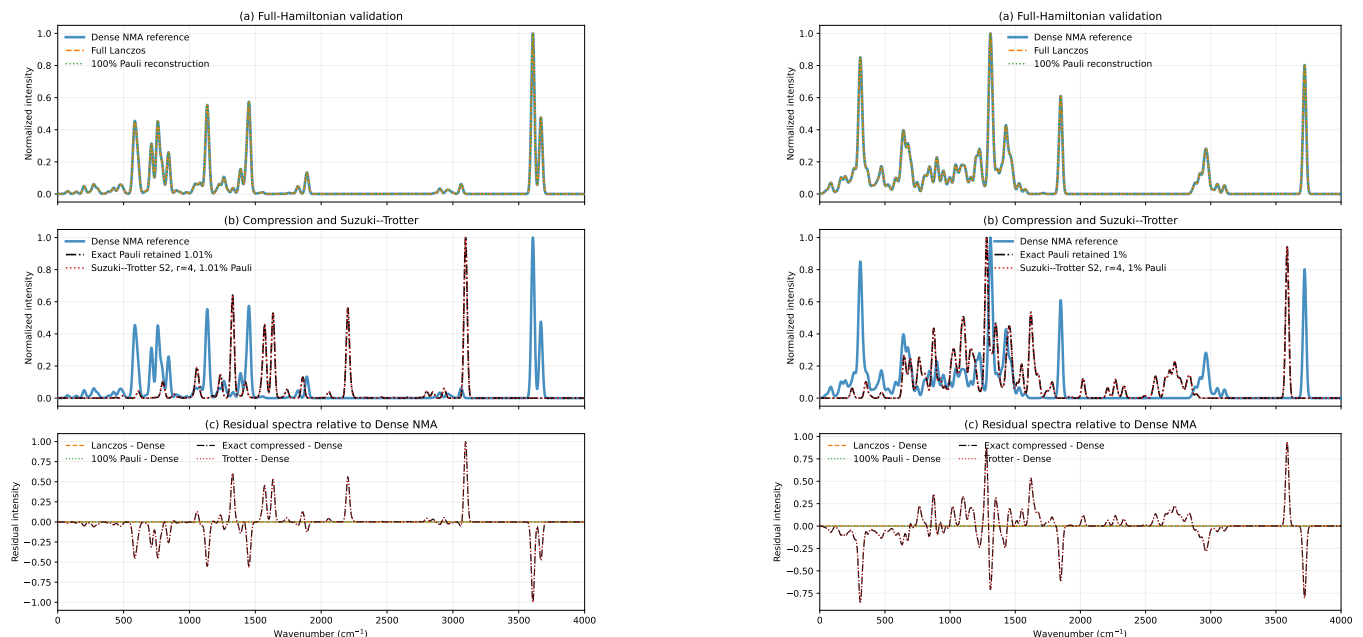


FIG. 5. Representative validation plots for Citrinin (left) and T-2 toxin (right). For each molecule, (a) Dense NMA, full Lanczos, and the 100% Pauli reconstruction overlap in the full-Hamiltonian validation; (b) extreme 1%–1.01% Pauli retention strongly modifies the spectrum, whereas the exact-compressed and Suzuki–Trotter results remain closely paired; and (c) residuals relative to Dense NMA are negligible for the full-Hamiltonian checks but large and nearly coincident for the compressed and Trotter results. The repeated pattern identifies aggressive compression as the dominant source of the observed spectral distortion.

CSV.

E. Interactive normal-mode interpretation

Each molecule can optionally generate a browser-based 3D viewer using 3Dmol.js¹⁹ with molecular rotation, zoom, representation controls, normal-mode animation, and clickable Dense-NMA IR-mode markers. The viewer is a visualization layer rather than part of the numerical validation. Its purpose is to connect a selected spectral feature to the corresponding Cartesian displacement pattern and to facilitate qualitative inspection of mode mixing under compression. A future quantitative extension should report mode-shape overlap, for example the modal assurance criterion

$$\text{MAC}_{ij} = \frac{|\mathbf{v}_i^T \mathbf{v}_j|^2}{(\mathbf{v}_i^T \mathbf{v}_i)(\mathbf{v}_j^T \mathbf{v}_j)}. \quad (37)$$

VI. SOFTWARE ENGINEERING, LIMITATIONS, AND DISCUSSION

A. What the software adds beyond a feature list

The contribution of the program is not the individual use of Dense diagonalization, Lanczos iteration, Pauli decomposition, or a Suzuki product formula; each is established. The useful software contribution is the way

these components are connected with explicit validation boundaries and resource accounting. The implementation was designed so that an operator-transformation error, a compression error, and a propagation error cannot silently collapse into one reported “quantum” discrepancy. This design choice directly addresses the type of implementation detail that software papers often omit but that determines whether another group can reproduce or extend the calculation.

B. Algorithmic behavior revealed by the implementation

The principal result of this work is methodological rather than a claim of quantum speedup. The exact Pauli mapping is validated to machine precision, the classical eigensolution is independently checked, and the effect of Hamiltonian sparsification is separated from the effect of the product formula. This error decomposition is particularly important because an apparently poor “Trotter spectrum” can in fact be an accurate propagation of an already over-compressed Hamiltonian.

The Patulin case provides a concrete warning against coefficient-only sparsification criteria. Although the 10% representation retains approximately 99.5% of the squared coefficient norm, the spectral cosine drops to 0.514 and the frequency MAE exceeds 150 cm^{−1}. Nearby or coupled normal modes can be sensitive to matrix perturbations that appear small under a global coefficient norm; eigenvector

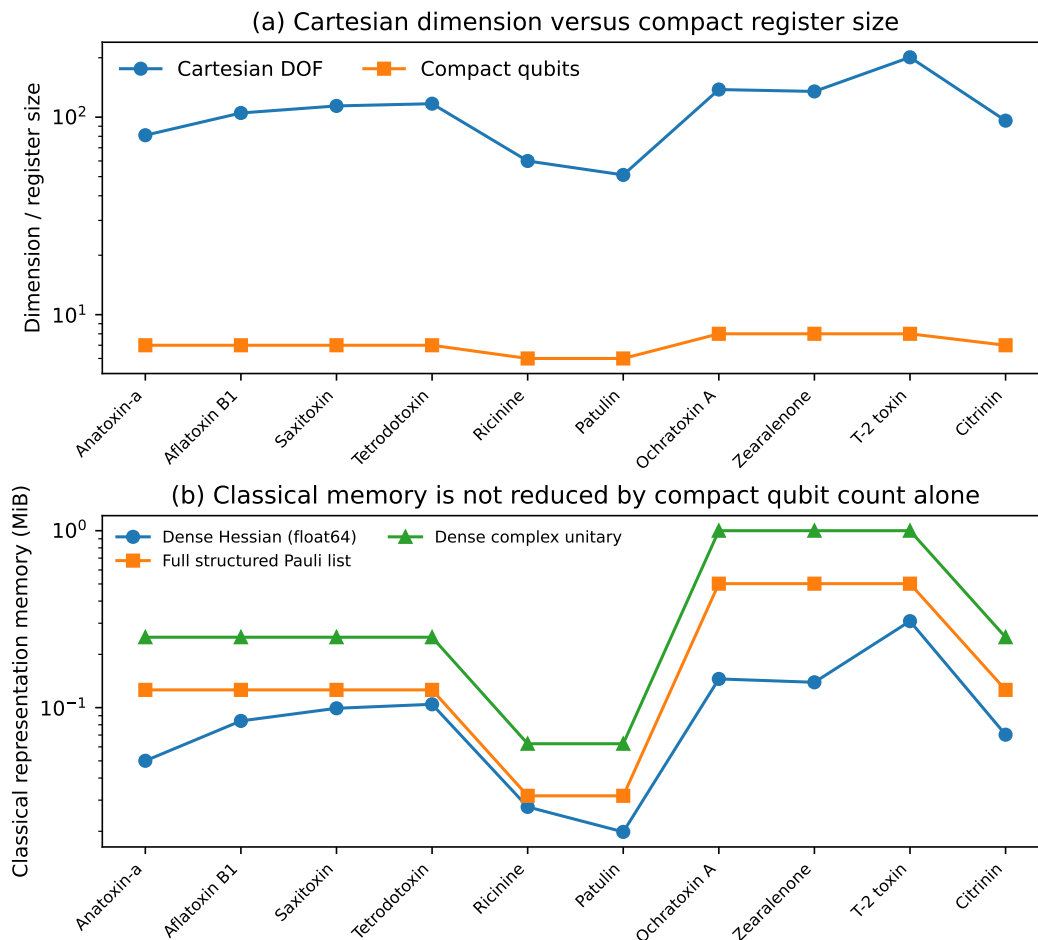


FIG. 6. Representation scaling for the ten-molecule benchmark. (a) Cartesian degrees of freedom and compact index-register qubits. (b) Analytic classical memory for the dense double-precision Hessian, the full structured Pauli list, and a dense complex unitary. The compact register is logarithmic in dimension, but the current classical Pauli and unitary representations do not inherit that memory reduction automatically.

rotation can then redistribute the dipole-displacement intensity. Consequently, a physically motivated compression strategy must constrain spectral observables directly, not merely W_1 or W_2 .

The compact index register is likewise easy to misinterpret. Encoding a d -dimensional real symmetric operator in $n_q = \lceil \log_2 d \rceil$ qubits is a meaningful state-space compression relative to direct one-hot coordinate encodings, and related compact encodings have been explored for vibrational quantum algorithms.^{4,20} However, the number of nonzero Pauli terms and the gate-level implementation remain essential. In the present full decomposition, P grows quadratically with the padded dimension, and the dense classical unitary used for validation grows as D^2 . Thus, the present work should be described as a *compact quantum-compatible encoding with explicit resource accounting*, not as a demonstrated memory or runtime advantage.

C. Interoperability with electronic-structure workflows

The numerical core is naturally positioned as a post-processing layer between an electronic-structure calculation and compact-operator analysis. In a production workflow, an upstream electronic-structure package should provide an optimized geometry, atomic masses, a Cartesian or mass-weighted Hessian, and either dipole derivatives or mode intensities. The current prototype does not yet claim native support for a particular quantum-chemistry file format. Adding and testing such interfaces is the most important step for the *Electronic Structure Software 2.0* submission because it converts the present self-contained benchmark into a reusable analysis component of an electronic-structure software ecosystem.

A practical first release should support at least one openly testable route, for example a documented generic NumPy/text matrix interface plus one direct parser or Python API for a widely used electronic-structure package. The benchmark should then be repeated for several

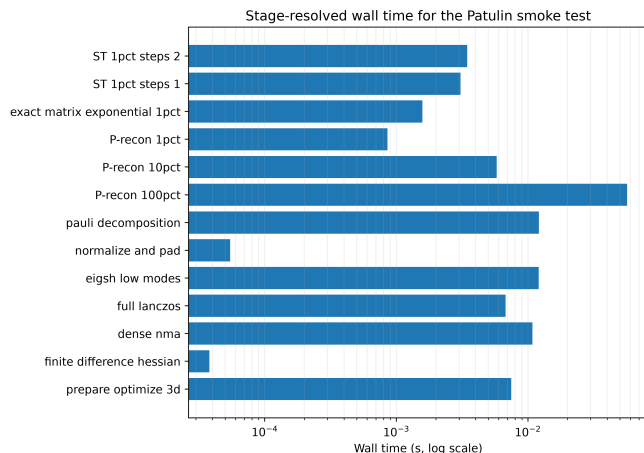


FIG. 7. Measured stage-resolved wall time for the Patulin smoke test. The horizontal axis is logarithmic. The result illustrates why classical Pauli preprocessing and dense-unitary emulation must be included in an end-to-end cost discussion rather than reporting only the propagation kernel.

molecules using a converged DFT Hessian and physical dipole derivatives. That comparison will test both numerical interoperability and the transfer of the compact Pauli analysis from the current force-field development backend to an electronic-structure source.

D. Limitations and required spectroscopy upgrade

The most important chemical-physics limitation is the spectroscopy backend. MMFF94s geometries and finite-difference Hessians are useful for fast algorithmic development,¹¹ but the Gasteiger fixed-charge intensity of Eq. (6) is not a physical *ab initio* IR intensity. Before a final spectroscopy-centered JCP submission, a representative subset should be recalculated with a quantum-chemical Hessian and dipole derivatives (e.g., a converged DFT protocol), and the resulting frequencies/intensities should be compared with the compact Pauli pipeline. The exact Pauli and Trotter machinery can remain unchanged because it consumes the final real symmetric vibrational operator and mode-intensity information rather than depending on MMFF-specific physics.

A second limitation is that the present benchmark is harmonic. Anharmonic vibrational structure is precisely where quantum algorithms may become more compelling, and prior work has considered modal, bosonic, and explicitly anharmonic formulations.^{17,18,20,21} Extending the current accuracy–resource methodology to anharmonic Hamiltonians would therefore be a natural next step.

A third limitation is the direct classical Pauli decomposition and dense-unitary emulation. These were chosen for transparent verification, not scalability. For larger operators, matrix-free coefficient generation, symmetry filtering, commuting-group structure, sparse propagation, or gate-

level cost estimates will be necessary. Product-formula resource claims should ultimately be reported with Pauli weights and commutator-aware error bounds.^{17,18,22}

E. Scaling from molecular benchmarks to fixed-rank protein calculations

The molecular benchmarks establish that compact binary indexing can represent vibrational Hamiltonians with a qubit register that grows logarithmically with the dimension of the encoded space. To examine how this representation behaves when the underlying molecular system becomes substantially larger, we extended the workflow to a controlled protein-scale benchmark. Figure 8 separates four quantities that should not be conflated: the physical Cartesian dimension, the measured classical runtime, the process-level memory footprint, and the size of the compact qubit register.

Panel 8(a) establishes the physical size range of the protein benchmark. The Cartesian dimension increases from $d = 912$ for 1L2Y to $d = 5568$ for 1RBX, with intermediate dimensions of 1788, 1926, 2565, and 3693 for 1VII, 1CRN, 1PGB, and 1UBQ, respectively. Thus, the largest system contains more than six times as many Cartesian degrees of freedom as the smallest. This physical dimension is the relevant input scale for operations performed before projection onto the reduced modal space and should therefore be distinguished from the dimension of the final compact Hamiltonian.

Panel 8(b) shows that this increase in physical size remains visible in the classical computational cost. The measured end-to-end wall time increases from 18.2 s for 1L2Y to 115.8 s for 1RBX. The intermediate measurements are 23.9 s for 1VII, 35.4 s for 1CRN, 35.6 s for 1PGB, and 52.7 s for 1UBQ. The overall trend is therefore strongly increasing with molecular dimension, although it is not expected to follow a single simple power law over this limited six-system benchmark because the total workflow combines computational stages with different scaling behavior and fixed software overheads.

The stage-resolved bars provide additional information about the origin of this cost. They show the contributions from the matrix-free low-mode solve, dense Hessian construction, and dense eigensolution used for reference and validation. These displayed components become increasingly important for larger proteins, and the increase is particularly evident for 1RBX. The red diamonds, however, correspond to independently measured end-to-end wall times. They should not be interpreted as the arithmetic sum of only the three stacked components because molecular preparation, system construction, transformations, validation, telemetry, file operations, and other workflow stages also contribute to the total elapsed time.

This behavior is consistent with the computational structure of the workflow. For a Cartesian problem of dimension d , central finite-difference Hessian construction requires a number of force or gradient evaluations that

Protein-Scale Scalability of the Vibrational Hamiltonian Workflow

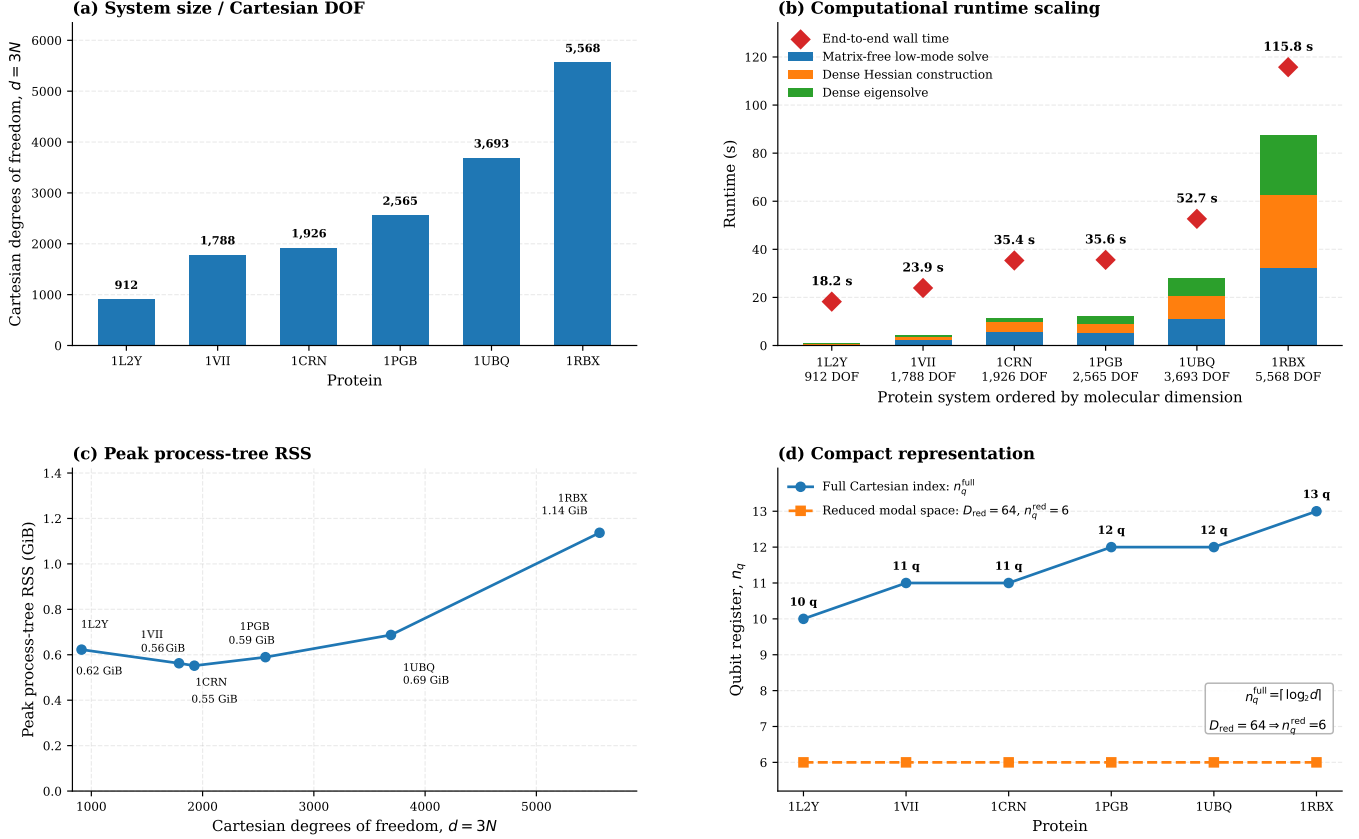


FIG. 8. Protein-scale scalability of the computational workflow. **(a)** System size expressed as the Cartesian dimension, $d = 3N$, for each protein in the benchmark set. **(b)** Computational runtime scaling for the same protein systems. The red diamonds indicate the measured end-to-end wall time, while the stacked bars show the contributions from the matrix-free low-mode solve, dense Hessian construction, and dense eigensolve stages. **(c)** Peak process-tree resident-set (RSS) as a function of the Cartesian dimension, showing the memory requirements as the protein size increases. **(d)** Compact representation of the protein systems. The full Cartesian index requires $n_q^{\text{full}} = \lceil \log_2 d \rceil$ qubits, whereas projection onto the fixed reduced modal space, $D_{\text{red}} = 64$, requires $n_q^{\text{red}} = 6$ qubits for all protein systems. The six-qubit representation refers exclusively to the selected reduced modal subspace and not to the complete Cartesian Hessian of the protein.

grows with d , while explicit dense diagonalization carries a nominal $\mathcal{O}(d^3)$ asymptotic cost. Consequently, reducing the final Hamiltonian to a fixed modal dimension does not eliminate the classical work required to obtain and validate that reduced representation from the original molecular system.

Panel 8(c) shows a related trend in memory consumption. Peak process-tree RSS is approximately 0.62, 0.56, 0.55, 0.59, 0.69, and 1.14 GiB for 1L2Y, 1VII, 1CRN, 1PGB, 1UBQ, and 1RBX, respectively. The RSS values are not strictly monotonic for the smaller systems. This is expected because process-tree RSS is an empirical process-level measurement rather than the analytic storage size of a single matrix: it includes the Python runtime, loaded numerical libraries, molecular-mechanics infrastructure, temporary arrays, memory-allocation behavior, and other transient resources. The important observation is therefore not a presumed monotonic scaling law, but that the

largest physical system also produces the largest measured memory footprint, reaching approximately 1.14 GiB.

Panel 8(d) provides the complementary representation view. If the complete Cartesian spaces were indexed directly, the compact register would be

$$n_q^{\text{full}} = \lceil \log_2 d \rceil. \quad (38)$$

For the dimensions considered here, this gives 10 qubits for 1L2Y, 11 qubits for 1VII and 1CRN, 12 qubits for 1PGB and 1UBQ, and 13 qubits for 1RBX. Thus, while the physical Cartesian dimension increases by more than a factor of six, the corresponding direct compact index register increases from only 10 to 13 qubits. This logarithmic behavior is a property of the compact binary representation and not a statement about the computational complexity of constructing the underlying molecular Hamiltonian.

The protein calculations used in the present scaling experiment employ a different encoded object. Rather than constructing a Pauli representation of the complete Cartesian Hessian, the workflow projects the problem onto a fixed modal subspace with

$$D_{\text{red}} = 64, \quad n_q^{\text{red}} = \lceil \log_2 D_{\text{red}} \rceil = 6. \quad (39)$$

The dashed line in Fig. 8(d) therefore remains at six qubits for all six proteins. This result should be interpreted carefully. The six-qubit register is not a lossless six-qubit encoding of the complete $3N$ -dimensional protein Hessian. It encodes the selected 64-dimensional reduced modal Hamiltonian. The increasing separation between n_q^{full} and n_q^{red} illustrates the representational benefit of fixed-rank modal projection, but it does not establish that $D_{\text{red}} = 64$ is sufficient for every vibrational observable or every protein.

A physically meaningful reduced representation therefore requires a separate rank-convergence analysis in which D_{red} is systematically increased and the relevant frequencies, mode shapes, spectral observables, or dynamical quantities are monitored for convergence. The present protein series should accordingly be interpreted as a controlled fixed-rank scalability test, rather than as evidence that 64 modes universally replace full normal-mode analysis.

Taken together, the four panels expose an important separation between *representational scaling* and *computational scaling*. The physical problem grows from 912 to 5568 Cartesian degrees of freedom, whereas direct compact indexing grows only from 10 to 13 qubits and the deliberately fixed-rank reduced Hamiltonian remains at six qubits. At the same time, end-to-end runtime rises from 18.2 to 115.8 s and the largest measured process-tree RSS reaches approximately 1.14 GiB. Compactness of the final encoded Hamiltonian therefore does not make the complete computational workflow independent of molecular size.

The protein benchmark consequently separates three resource layers: (i) the physical molecular dimension $d = 3N$; (ii) the encoded Hilbert-space dimension, characterized here by n_q^{full} or n_q^{red} ; and (iii) the classical resources required to construct, project, validate, and analyze the representation. Compact binary indexing acts directly on the second layer, whereas the first and third remain coupled through the classical algorithms used upstream of the compact representation. The present results should therefore be regarded as evidence of *representational compactness*, not as a demonstration of end-to-end computational or quantum speedup. Any future resource-advantage claim must account for the complete pipeline, including reduced-subspace construction, Hamiltonian generation, Pauli decomposition, state preparation, circuit synthesis, measurement, and the accuracy required for the target vibrational observable.

VII. CONCLUSIONS

We developed a reproducible software workflow for compact Pauli analysis of molecular vibrational Hamiltonians, with particular emphasis on identifying the numerical stage at which an approximation begins to alter the physical result. The workflow combines molecular preparation, harmonic normal-mode analysis, Dense/Lanczos validation, compact power-of-two embedding, exact Pauli decomposition, controlled sparsification, same-Hamiltonian Suzuki–Trotter validation, and stage-resolved runtime and memory telemetry.

Across the molecular benchmark, the exact compact mapping reproduces the embedded vibrational operators to machine precision. The subsequent compression tests show, however, that retaining most of the squared Pauli-coefficient weight does not necessarily preserve the vibrational spectrum. In the stress tests examined here, the Suzuki–Trotter approximation follows the exact dynamics of the same compressed Hamiltonian more closely than the compressed Hamiltonian reproduces the original Dense-NMA spectrum. This separation identifies Hamiltonian compression, rather than the product formula itself, as the dominant source of error in those cases and motivates compression criteria that are sensitive to the target vibrational observables.

The protein-scale extension provides a complementary view of what compact encoding does, and does not, reduce. Across the six protein systems considered here, the physical Cartesian dimension increases from 912 to 5568 degrees of freedom. Direct compact indexing of these full Cartesian spaces would require only 10–13 qubits because the register size grows logarithmically with the encoded dimension. The calculations reported here, however, use a fixed modal subspace with $D_{\text{red}} = 64$, so that the reduced Hamiltonian requires six qubits for every protein. This six-qubit register represents the selected 64-dimensional modal subspace and must not be interpreted as a lossless encoding of the complete protein Hessian.

The runtime and memory measurements make this distinction explicit. Although the reduced encoded dimension remains fixed, the measured end-to-end runtime increases from 18.2 to 115.8 s across the protein series, while the largest peak process-tree RSS reaches approximately 1.14 GiB. The stage-resolved measurements further show that matrix-free low-mode extraction, dense Hessian construction, and dense eigensolution retain a dependence on the size of the underlying molecular system. Compactness of the final encoded Hamiltonian therefore does not make the complete computational workflow independent of protein size.

These results separate three resource layers that are easily conflated: the physical molecular dimension, the dimension of the encoded Hilbert space, and the classical resources required to construct and validate that representation. Compact binary indexing acts directly on the second layer, whereas the first and third remain connected through the classical algorithms used upstream

of the encoding. The protein results should therefore be interpreted as evidence of *representational compactness* and fixed-rank scalability, rather than as evidence of end-to-end computational or quantum speedup.

The fixed $D_{\text{red}} = 64$ protein calculations also define an important encoded modal dimension is held constant while the physical system grows, but they do not establish that 64 modes are sufficient for every protein or every vibrational observable. That question requires systematic rank-convergence studies in which D_{red} is increased and the relevant frequencies, mode shapes, spectral quantities, or dynamical observables are monitored for convergence.

Future developments should therefore proceed along two complementary directions. From a physical perspective, the current validation backend can be coupled to electronic-structure Hessians and physical dipole derivatives, whereas the protein extension can be evaluated across increasing reduced-space ranks. From a representation perspective, compression strategies must extend beyond coefficient magnitude alone to incorporate observable-aware error criteria. Consequently, any rigorous assessment of quantum computational advantage must evaluate the complete computational pipeline, including reduced-subspace construction, Hamiltonian generation, Pauli decomposition, state preparation, circuit synthesis, measurement, and the target observable precision.

Overall, the present workflow provides a controlled bridge between conventional vibrational analysis and quantum-compatible Hamiltonian representations. Its main result is not that a small qubit register removes the computational cost of a large molecular problem, but that the different sources of numerical error and resource consumption can be isolated and measured independently. This distinction provides a more rigorous basis for evaluating when compact Pauli representations are physically accurate, computationally useful, and ultimately suitable for larger-scale molecular and protein applications.

SUPPLEMENTARY MATERIAL

The Supplementary Material contains detailed stage-resolved timing and RSS output, Patulin compression and Trotter CSV tables, additional representative spectra, and a separate protein scaling stress test with explicit convergence guardrails.

DATA AVAILABILITY

The current development archive contains the ten-molecule manifest and the machine-readable tables used for the software-validation examples. Before submission, these files should be deposited with the versioned software release or in a persistent repository and the DOI/URL inserted here.

CODE AVAILABILITY

The source code, environment specifications, installation instructions, automated tests, and example inputs/outputs required to reproduce the results and figures presented in this work are publicly available on the GitHub repository at https://github.com/eduardo45pr/sptmgp-NMA-Krylov-Suzuki-Work1_30-09-2026

AUTHOR DECLARATIONS

Conflict of Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Author Contributions

Conceptualisation, S.P.T. methodology, S.P.T.; validation, S.P.T. and E.P.E.R.; formal analysis, S.P.T. and E.P.E.R.; investigation, S.P.T. and E.P.E.R.; resources, S.P.T. and D.R.A.Z.; data curation, E.P.E.R. and S.P.T.; writing—original draft preparation, S.P.T.; writing—review and editing S.P.T. and E.P.E.R.; visualization, E.P.E.R. and S.P.T.; supervision, S.P.T. and D.R.A.Z.; project administration, S.P.T. and D.R.A.Z.; funding acquisition S.P.T. All authors have read and agreed to the published version of the manuscript.

Funding

This research was fully funded by SIT HEALTH SAS. The funder provided 100% of the financial support for this work.

REFERENCES

- ¹C. David Sherrill, David E. Manolopoulos, Todd J. Martinez, and Angelos Michaelides. Electronic structure software. *The Journal of Chemical Physics*, 153:070401, 2020.
- ²C. David Sherrill, David E. Manolopoulos, Todd J. Martinez, Michele Ceriotti, and Angelos Michaelides. Chemical physics software. *The Journal of Chemical Physics*, 155:010401, 2021.
- ³Sam McArdle, Alexander Mayorov, Xiao Shan, Simon C. Benjamin, and Xiao Yuan. Digital quantum simulation of molecular vibrations. *Chemical Science*, 10:5725–5735, 2019.
- ⁴Nicolas P. D. Sawaya, Tim Menke, Thi Ha Kyaw, Sonika Johri, Alan Aspuru-Guzik, and Gian Giacomo Guerreschi. Resource-efficient digital quantum simulation of d-level systems for photonic, vibrational, and spin-s hamiltonians. *npj Quantum Information*, 6:49, 2020.
- ⁵Ryan Babbush, Jarrod McClean, Dave Wecker, Alan Aspuru-Guzik, and Nathan Wiebe. Chemical basis of Trotter–Suzuki errors in quantum chemistry simulation. *Physical Review A*, 91:022311, 2015.

- ⁶William M. Kirby, S. Hadi, Michael Kreshchuk, and Peter J. Love. Quantum simulation of second-quantized hamiltonians in compact encoding. *Physical Review A*, 104:042607, 2021.
- ⁷Mario Motta, Erika Ye, Jarrod R. McClean, Zhendong Li, Austin J. Minnich, Ryan Babbush, and Garnet Kin-Lic Chan. Low rank representations for quantum simulation of electronic structure. *npj Quantum Information*, 7:83, 2021.
- ⁸Cornelius Lanczos. An iteration method for the solution of the eigenvalue problem of linear differential and integral operators. *Journal of Research of the National Bureau of Standards*, 45:255–282, 1950.
- ⁹Beresford N. Parlett. *The Symmetric Eigenvalue Problem*. Prentice-Hall, Englewood Cliffs, NJ, 1980.
- ¹⁰P. J. J. O’Malley, Ryan Babbush, Ian D. Kivlichan, Jonathan Romero, Jarrod R. McClean, et al. Scalable quantum simulation of molecular energies. *Physical Review X*, 6:031007, 2016.
- ¹¹Thomas A. Halgren. Merck molecular force field. i. basis, form, scope, parameterization, and performance of MMFF94. *Journal of Computational Chemistry*, 17:490–519, 1996.
- ¹²Thomas A. Halgren. MMFF VI. MMFF94s option for energy minimization studies. *Journal of Computational Chemistry*, 20:720–729, 1999.
- ¹³Paolo Tosco, Nikolaus Stiefl, and Gregory Landrum. Bringing the MMFF force field to the RDKit: implementation and validation. *Journal of Cheminformatics*, 6:37, 2014.
- ¹⁴Johann Gasteiger and Mario Marsili. Iterative partial equalization of orbital electronegativity—a rapid access to atomic charges. *Tetrahedron*, 36:3219–3228, 1980.
- ¹⁵Hale F. Trotter. On the product of semi-groups of operators. *Proceedings of the American Mathematical Society*, 10:545–551, 1959.
- ¹⁶Masuo Suzuki. Generalized trotter’s formula and systematic approximants of exponential operators and inner derivations with applications to many-body problems. *Communications in Mathematical Physics*, 51:183–190, 1976.
- ¹⁷D. Tenev, P. J. Ollitrault, S. M. Harwood, T. P. Gujarati, S. Raman, A. Mezzacapo, and S. Mostame. Refining resource estimation for the quantum computation of vibrational molecular spectra through trotter error analysis. *Quantum*, 9:1630, 2025.
- ¹⁸S. Malpathak, S. D. Kallullathil, I. Loaiza, S. Fomichev, J. M. Arrazola, and A. F. Izmaylov. Trotter simulation of vibrational hamiltonians on a quantum computer. *Journal of Chemical Theory and Computation*, 22:95–119, 2026.
- ¹⁹Nicholas Rego and David Koes. 3Dmol.js: Molecular visualization with WebGL. *Bioinformatics*, 31:1322–1324, 2015.
- ²⁰Nicolas P. D. Sawaya, Francesco Paesani, and Daniel P. Tabor. Near- and long-term quantum algorithmic approaches for vibrational spectroscopy. *Physical Review A*, 104:062419, 2021.
- ²¹Pauline J. Ollitrault, Alberto Baiardi, Markus Reiher, and Ivano Tavernelli. Hardware efficient quantum algorithms for vibrational structure calculations. *Chemical Science*, 11:6842–6855, 2020.
- ²²Andrew M. Childs, Yuan Su, Minh C. Tran, Nathan Wiebe, and Shuchen Zhu. Theory of trotter error with commutator scaling. *Physical Review X*, 11:011020, 2021.